

train_phase_classifier

January 19, 2026

1 Human-Embryo *Phase Classifier* — End-to-End Notebook

This notebook trains a deep network that maps a **single time-lapse frame** to its *morphokinetic phase* (t_2 , t_3 , ..., t_B).

1.1 Paths

```
PROJECT_ROOT      = Path("..")
ANNOT_DIR         = PROJECT_ROOT / 'data' / 'embryo_dataset_annotations'
```

```
[1]: # @title Install & Imports

import random, re, math, json, os, gc
from pathlib import Path
from collections import defaultdict

import numpy as np
import pandas as pd
from PIL import Image
from tqdm.auto import tqdm

import torch, torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.transforms.v2 as v2
import torchvision.models as models

from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

```
/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
[2]: # @title Paths & Device
PROJECT_ROOT      = Path("..").resolve()
FOCAL_PLANES      = ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']
DATA_ROOTS        = {fp: PROJECT_ROOT / 'data' / 'embryo_dataset_silver' / fp_
    ↪for fp in FOCAL_PLANES}
ANNOT_DIR          = PROJECT_ROOT / 'data' / 'embryo_dataset_annotations'

# Verify all focal plane directories exist
for fp, dr in DATA_ROOTS.items():
    assert dr.is_dir(), f"{dr} not found"
assert ANNOT_DIR.is_dir(), f"{ANNOT_DIR} not found"

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Running on", device)
print(f"Focal planes: {FOCAL_PLANES}")
```

Running on cuda

Focal planes: ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']

```
[3]: # @title Embryo Exclusion List
EXCLUDE_EMBRYOS = {
    "BJ492-8",
    "ALR493-6"
}
```

```
[4]: # @title Helpers

PHASES = [
    "tPB2", "tPNa", "tPNf",
    "t2", "t3", "t4", "t5", "t6", "t7", "t8", "t9+",
    "tM", "tSB", "tB", "tEB", "tHB"
]
PHASE2IDX = {p: i for i, p in enumerate(PHASES)}
IDX2PHASE = {i: p for p, i in PHASE2IDX.items()}
NUM_CLASSES = len(PHASES)

def extract_frame_idx(name: str):
    m = re.search(r"(\d+)\.jpeg$", name)
    return int(m.group(1)) if m else None

def index_frames(folder: Path):
    mapping = {}
    for fp in folder.glob("*.jpeg"):
        m = re.search(r"(\d+)\.jpeg$", fp.name)
        if m:
            mapping[int(m.group(1))] = fp
    return mapping
```

```

[5]: # @title PhaseDataset (filters empty frames)

FRAMES_PER_PHASE = 7      # None → keep all

class PhaseDataset(Dataset):
    def __init__(self, embryo_ids, transform, focal_planes=None,
        →frames_per_phase=10):
        self.transform = transform
        self.focal_planes = focal_planes or list(DATA_ROOTS.keys())
        self.samples = []      # [(img_path, label_idx), ...]

        for eid in tqdm(embryo_ids):
            csv_path = ANNOT_DIR / f"{eid}_phases.csv"
            if not csv_path.is_file():      # skip missing
                print(f"Warning: missing annotation for {eid}, skipping")
                continue

            df = pd.read_csv(csv_path, header=None,
                names=["phase", "start", "end"])

            # Process each focal plane for this embryo
            for fp in self.focal_planes:
                data_root = DATA_ROOTS[fp]
                frame2path = index_frames(data_root / eid)

                for phase, start, end in df.itertuples(index=False):
                    phase = str(phase).strip()
                    if phase not in PHASE2IDX:      # unknown label → skip
                        continue

                    # ---- subsample frames -----
                    if frames_per_phase and (end - start + 1) >
        →frames_per_phase:
                        sel = np.linspace(start, end, frames_per_phase,
        →dtype=int)

                        else:
                            sel = range(start, end + 1)

                    paths = [frame2path.get(f) for f in sel if frame2path.
        →get(f)]

                    lbl = PHASE2IDX[phase]
                    for p in paths:
                        self.samples.append((p, lbl))

            print(f"Got {len(self.samples):,} frames from {len(self.focal_planes)}
        →focal planes")

```

```

        self.num_classes = NUM_CLASSES

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, i):
        path, label = self.samples[i]
        img = Image.open(path).convert("L")
        return self.transform(img), torch.tensor(label)

```

```

[6]: # @title Transforms & Data Split
MAX_EMBRYOS = None # 25 for a quick run

IMG_SIZE = 256
# Common transforms for all sets (resize + normalize)
base_tfm = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.ToTensor(),
    T.Normalize([0.5], [0.5]), # normalize all sets consistently
])

# Additional augmentations only for training
train_aug = v2.Compose([
    v2.ColorJitter(
        brightness=0.25,
        contrast=0.25,
    ),
    v2.RandomApply([
        v2.RandomAdjustSharpness(sharpness_factor=1.2)
    ], p=0.15),
    v2.RandomApply([
        v2.GaussianBlur(kernel_size=(3,3), sigma=(0.1, 0.6))
    ], p=0.15),
    v2.RandomAffine(
        degrees=180,
        translate=(0.02, 0.02),
        scale=(0.95, 1.05),
        shear=(-2, 2)
    ),
])

# Training pipeline: base transforms -> augmentations -> normalize
train_tfm = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.ToTensor(),
    train_aug,
    T.Normalize([0.5], [0.5])

```

```

])

# Val/Test pipeline: just the base transforms including normalization
val_tfm = base_tfm

```

```

[7]: # @title Diffusion Noise Schedule (for noise-aware classifier)
TIMESTEPS = 1000
BETA_START, BETA_END = 1e-4, 2e-2

betas = torch.linspace(BETA_START, BETA_END, TIMESTEPS, device=device)
alphas = 1.0 - betas
alphas_cumprod = torch.cumprod(alphas, dim=0)
sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alphas_cumprod)

# Avoid the noisiest tail of timesteps for classifier training
NOISE_T_MAX = max(1, int(0.9 * TIMESTEPS))

def q_sample(x0: torch.Tensor, t: torch.Tensor, noise: torch.Tensor | None = None) → None:
    """Add diffusion noise to x0 at timestep t."""
    if noise is None:
        noise = torch.randn_like(x0)
    sqrt_alpha = sqrt_alphas_cumprod[t].view(-1, 1, 1, 1)
    sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t].view(-1, 1, 1, 1)
    return sqrt_alpha * x0 + sqrt_one_minus * noise, noise

def sample_timesteps(batch_size: int):
    # sample from [0, NOISE_T_MAX) to avoid extreme noise
    return torch.randint(0, NOISE_T_MAX, (batch_size,), device=device, dtype=torch.long)

```

```

[8]: # embryo-level split
all_embryos = sorted([p.name for p in DATA_ROOTS[FOCAL_PLANES[0]].iterdir() if p.is_dir()
                      and p.name not in EXCLUDE_EMBRYOS])

random.seed(42)
np.random.seed(42)
random.shuffle(all_embryos)

# respect MAX_EMBRYOS
if MAX_EMBRYOS is not None:
    assert MAX_EMBRYOS >= 3, "Need 3 embryos to form train/val/test"
    all_embryos = all_embryos[:MAX_EMBRYOS]

n_val = max(1, int(0.10*len(all_embryos)))

```

```

n_test = max(1, int(0.10*len(all_embryos)))
val_embryos = all_embryos[:n_val]
test_embryos = all_embryos[n_val:n_val+n_test]
train_embryos = all_embryos[n_val+n_test:]

# Ensure class-wise coverage in VAL/TEST by pulling missing-phase embryos
↳from TRAIN
import pandas as pd

# Cache: embryo -> set(phases present in its CSV)
def load_embryo_phases(eid: str):
    csv_path = ANNOT_DIR / f"{eid}_phases.csv"
    if not csv_path.is_file():
        return set()
    df = pd.read_csv(csv_path, header=None, names=["phase", "start", "end"])
    return {
        str(p).strip()
        for p in df["phase"].tolist()
        if str(p).strip() in PHASE2IDX
    }

embryo2phases = {eid: load_embryo_phases(eid) for eid in all_embryos}

def split_phase_coverage(eids):
    covered = set()
    for eid in eids:
        covered |= embryo2phases.get(eid, set())
    return covered

def enforce_coverage_for(split_name, split_list, train_list):
    added = []
    covered = split_phase_coverage(split_list)
    for phase in PHASES:
        if phase not in covered:
            # find a donor embryo in TRAIN that has this phase
            donor = next((eid for eid in train_list
                           if phase in embryo2phases.get(eid, set())
                           and eid not in split_list), None)
            if donor is not None:
                train_list.remove(donor)
                split_list.append(donor)
                added.append((phase, donor))
                covered |= embryo2phases.get(donor, set())
            else:
                print(f"[warn] No available TRAIN embryo with phase '{phase}'")
                ↳to add to {split_name}."
    if added:

```

```

        print(f"[info] Added to {split_name}: " + ", ".join([f"{ph}->{eid}" for
        ↪ph, eid in added]))
    else:
        print(f"[info] {split_name} already covers all phases.")

# Apply to VAL then TEST (only pull from TRAIN)
enforce_coverage_for("VAL", val_embryos, train_embryos)
enforce_coverage_for("TEST", test_embryos, train_embryos)

# (Optional) quick sanity check
for name, split in [("TRAIN", train_embryos), ("VAL", val_embryos), ("TEST",
        ↪test_embryos)]:
    cov = split_phase_coverage(split)
    missing = [p for p in PHASES if p not in cov]
    print(f"{name}: {len(split)} embryos | missing phases: {missing if missing
        ↪else 'None'}")

print(f"Embryos  train {len(train_embryos)} | val {len(val_embryos)} | test
        ↪{len(test_embryos)}")

```

```

[info] Added to VAL: tHB->LS123-3
[info] TEST already covers all phases.
TRAIN: 561 embryos | missing phases: None
VAL: 71 embryos | missing phases: None
TEST: 70 embryos | missing phases: None
Embryos  train 561 | val 71 | test 70

```

```

[9]: train_ds = PhaseDataset(train_embryos, transform=train_tfm,
        ↪frames_per_phase=FRAMES_PER_PHASE)
    val_ds   = PhaseDataset(val_embryos,   transform=val_tfm,
        ↪frames_per_phase=FRAMES_PER_PHASE)
    test_ds  = PhaseDataset(test_embryos,  transform=val_tfm,
        ↪frames_per_phase=FRAMES_PER_PHASE)

    from collections import Counter
    train_labels = [lbl for _, lbl in train_ds.samples]
    class_counts = Counter(train_labels)
    num_classes  = train_ds.num_classes

    # Weighted sampler for TRAIN to help rare phases
    import numpy as np
    from torch.utils.data import DataLoader, WeightedRandomSampler

    sample_w = np.array([1.0 / class_counts[lbl] for _, lbl in train_ds.samples],
        ↪dtype=np.float64)

```

```
sampler = WeightedRandomSampler(torch.from_numpy(sample_w),
    ↳ num_samples=len(sample_w), replacement=True)

BATCH = 32
train_dl = DataLoader(train_ds, batch_size=BATCH, sampler=sampler,
    ↳ num_workers=8, pin_memory=True)
val_dl = DataLoader(val_ds, batch_size=BATCH, shuffle=False, num_workers=8,
    ↳ pin_memory=True)
test_dl = DataLoader(test_ds, batch_size=BATCH, shuffle=False, num_workers=8,
    ↳ pin_memory=True)
```

100%| | 561/561 [00:26<00:00, 21.17it/s]

Got 289,387 frames from 7 focal planes

100%| | 71/71 [00:03<00:00, 21.12it/s]

Got 37,282 frames from 7 focal planes

100%| | 70/70 [00:03<00:00, 18.83it/s]

Got 36,630 frames from 7 focal planes

```
[10]: # Class Distribution Visualization
from collections import Counter
import matplotlib.pyplot as plt

# Count samples per class
train_counts = Counter([lbl for _, lbl in train_ds.samples])
val_counts = Counter([lbl for _, lbl in val_ds.samples])
test_counts = Counter([lbl for _, lbl in test_ds.samples])

# Create a DataFrame for visualization
phase_names = [IDX2PHASE[i] for i in range(NUM_CLASSES)]
df_counts = pd.DataFrame({
    'Phase': phase_names,
    'Train': [train_counts.get(i, 0) for i in range(NUM_CLASSES)],
    'Val': [val_counts.get(i, 0) for i in range(NUM_CLASSES)],
    'Test': [test_counts.get(i, 0) for i in range(NUM_CLASSES)]
})

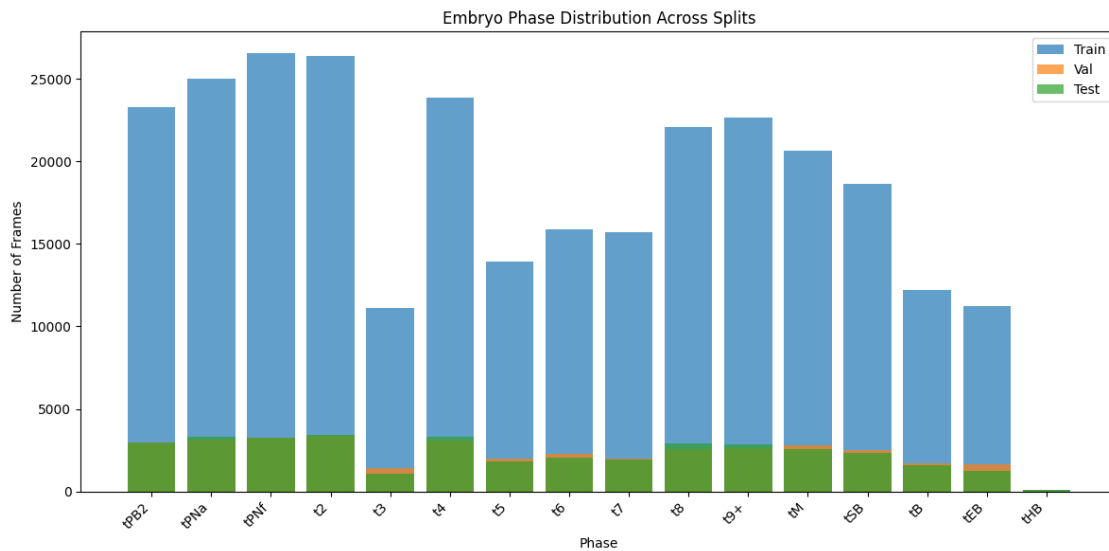
display(df_counts)

# Plot distributions
plt.figure(figsize=(12, 6))
plt.bar(df_counts['Phase'], df_counts['Train'], label='Train', alpha=0.7)
plt.bar(df_counts['Phase'], df_counts['Val'], label='Val', alpha=0.7)
plt.bar(df_counts['Phase'], df_counts['Test'], label='Test', alpha=0.7)
plt.xticks(rotation=45, ha='right')
plt.xlabel('Phase')
```



```
plt.ylabel('Number of Frames')
plt.title('Embryo Phase Distribution Across Splits')
plt.legend()
plt.tight_layout()
plt.show()
```

	Phase	Train	Val	Test
0	tPB2	23282	2947	2940
1	tPNa	25043	3136	3283
2	tPNf	26543	3262	3261
3	t2	26374	3359	3395
4	t3	11109	1435	1078
5	t4	23890	3080	3282
6	t5	13931	1988	1839
7	t6	15866	2247	2030
8	t7	15701	1997	1932
9	t8	22107	2533	2919
10	t9+	22686	2621	2870
11	tM	20663	2797	2562
12	tSB	18669	2505	2324
13	tB	12201	1695	1561
14	tEB	11259	1666	1260
15	tHB	63	14	94



```
[11]: # Embryo IDs per class (per split and overall)
```

```
from collections import defaultdict

def collect_embryo_ids_per_class(dataset):
```

```

phase_to_ids = defaultdict(set) # {label_idx: set(embryo_ids)}
for path, lbl in dataset.samples:
    eid = path.parent.name # folder name is the embryo id
    phase_to_ids[lbl].add(eid)
return phase_to_ids

train_ids = collect_embryo_ids_per_class(train_ds)
val_ids    = collect_embryo_ids_per_class(val_ds)
test_ids   = collect_embryo_ids_per_class(test_ds)

# Build "overall" as the union across splits
overall_ids = defaultdict(set)
for i in range(NUM_CLASSES):
    overall_ids[i] = set().union(train_ids.get(i, set()),
                                val_ids.get(i, set()),
                                test_ids.get(i, set()))

def print_ids(title, mapping):
    print(f"\n=== {title} ===")
    for i, phase in enumerate(PHASES):
        ids = sorted(mapping.get(i, set()))
        print(f"{phase} [{len(ids)} embryos]: {' '.join(ids) if ids else '-'}")

#print_ids("TRAIN embryo IDs per class", train_ids)
#print_ids("VAL embryo IDs per class", val_ids)
#print_ids("TEST embryo IDs per class", test_ids)
#print_ids("OVERALL embryo IDs per class (union)", overall_ids)

```

```

[12]: # @title Preview data-augmentations
import random, matplotlib.pyplot as plt
import torch

# -----
# helper to de-normalise a 1xHxW tensor
def denorm(x):
    return (x*0.5 + 0.5).clamp(0,1).squeeze().cpu().numpy()

# 1) Eight random augmented frames -----
N = 8
idxs = random.sample(range(len(train_ds)), N)
fig, axes = plt.subplots(1, N, figsize=(2.2*N, 2.5))

for ax, i in zip(axes, idxs):
    img, lbl = train_ds[i]
    ax.imshow(denorm(img), cmap='gray')
    ax.set_title(IDX2PHASE[lbl.item()], fontsize=7)
    ax.axis('off')

```

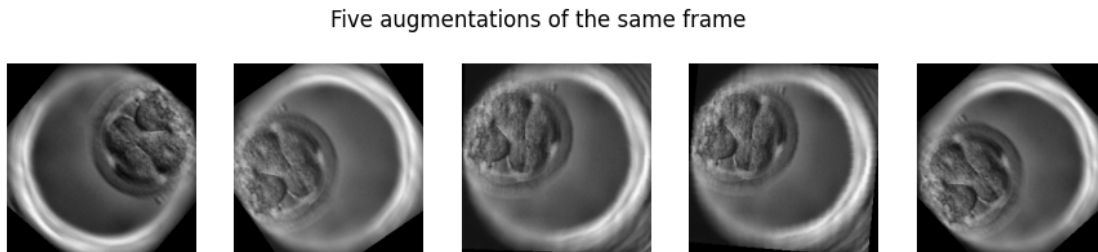
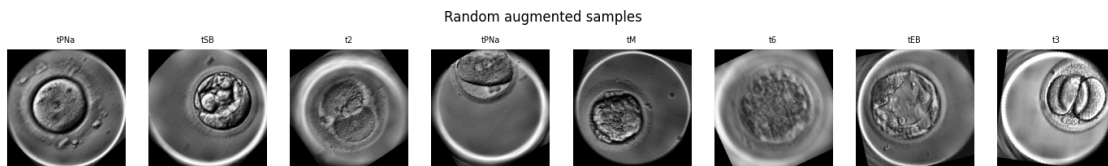
```

fig.suptitle("Random augmented samples", fontsize=12)
plt.show()

# 2) Same frame, 5 different augmentations -----
idx = random.choice(range(len(train_ds)))
base_path, _ = train_ds.samples[idx]
base_img = Image.open(base_path).convert("L")

fig, axes = plt.subplots(1, 5, figsize=(11, 2.4))
for ax in axes:
    aug = train_tfm(base_img)
    ax.imshow(denorm(aug), cmap='gray')
    ax.axis('off')
fig.suptitle("Five augmentations of the same frame", fontsize=12)
plt.show()

```



```

[13]: # @title Model (noise-aware + timestep conditioning)
from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights

class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim: int, max_period: float = 10000.0):
        super().__init__()
        self.dim = dim
        self.max_period = max_period
    def forward(self, t: torch.Tensor):
        # t: [B] int64 timesteps
        half = self.dim // 2
        t = t.float() / max(1, Timesteps - 1) # normalize to [0,1]

```

```

        freqs = torch.exp(
            -math.log(self.max_period) * torch.arange(0, half, device=t.device).
            ↪float() / half
        )
        args = t[:, None] * freqs[None] * 2 * math.pi
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=1)
        if self.dim % 2 == 1:
            emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], dim=1)
        return emb

class TimeCondEfficientNet(nn.Module):
    def __init__(self, n_classes: int, time_dim: int = 128):
        super().__init__()
        self.base = efficientnet_v2_s(weights=None)
        # Convert first layer to grayscale while preserving pre-trained weights
        old_conv = self.base.features[0][0]
        new_conv = nn.Conv2d(1, old_conv.out_channels, kernel_size=3, stride=2, ↪
            ↪padding=1, bias=False)
        with torch.no_grad():
            new_conv.weight.data = old_conv.weight.data.sum(dim=1, ↪
            ↪keepdim=True) / 3.0
        self.base.features[0][0] = new_conv

        in_feats = self.base.classifier[1].in_features
        self.base.classifier = nn.Identity()

        self.time_embed = SinusoidalTimeEmbedding(time_dim)
        self.time_mlp = nn.Sequential(
            nn.Linear(time_dim, time_dim),
            nn.SiLU(),
            nn.Linear(time_dim, time_dim),
        )

        self.classifier = nn.Sequential(
            nn.Dropout(p=0.2),
            nn.Linear(in_feats + time_dim, n_classes),
        )

    def forward(self, x: torch.Tensor, t: torch.Tensor):
        feats = self.base.features(x)
        feats = self.base.avgpool(feats)
        feats = torch.flatten(feats, 1)
        t_emb = self.time_mlp(self.time_embed(t))
        feats = torch.cat([feats, t_emb], dim=1)
        return self.classifier(feats)

n_classes = train_ds.num_classes

```

```

model = TimeCondEfficientNet(n_classes=n_classes, time_dim=128)
model.to(device)

optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-4)

from torch.optim.lr_scheduler import ReduceLROnPlateau
scheduler = ReduceLROnPlateau(
    optimizer, mode="min", factor=0.8, patience=2, min_lr=1e-7
)

# Loss with label smoothing
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

```

```

[14]: MIXUP_ALPHA, MIXUP_PROB = 0.6, 0.0
def apply_mixup(x, y, alpha=MIXUP_ALPHA):
    if alpha <= 0 or x.size(0) < 2: return x, (y, None, 1.0)
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0), device=x.device)
    return lam*x + (1-lam)*x[idx], (y, y[idx], lam)

```

```

[15]: EPOCHS = 1000
PATIENCE = 10 # Early stopping patience

# Mixed precision for speed
use_amp = torch.cuda.is_available()
scaler = torch.amp.GradScaler('cuda', enabled=use_amp)

best_loss = float('inf')
best_state = None
epochs_no_improve = 0
loss_history = []

for epoch in range(1, EPOCHS+1):
    # --- train ---
    model.train()
    tr_loss = 0.0
    tr_hits = 0
    for xb, yb in tqdm(train_dl, desc=f"Epoch {epoch}/{EPOCHS}"):
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad(set_to_none=True)

        do_mixup = np.random.rand() < MIXUP_PROB
        if do_mixup:
            xb, (y1, y2, lam) = apply_mixup(xb, yb)

        t = sample_timesteps(xb.size(0))
        xb_noisy, _ = q_sample(xb, t)

```

```

        with torch.autocast(device_type='cuda', enabled=use_amp):
            logits = model(xb_noisy, t)
            if do_mixup:
                loss = lam * criterion(logits, y1) + (1 - lam) *
↪criterion(logits, y2)
            else:
                loss = criterion(logits, yb)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        tr_loss += loss.item() * xb.size(0)
        tr_hits += (logits.argmax(1) == yb).sum().item()
    tr_loss /= len(train_ds)
    tr_acc = tr_hits / len(train_ds)

    # --- val ---
    model.eval()
    val_loss = 0.0
    val_hits = 0
    with torch.no_grad():
        for xb, yb in val_dl:
            xb, yb = xb.to(device), yb.to(device)
            t = sample_timesteps(xb.size(0))
            xb_noisy, _ = q_sample(xb, t)
            with torch.autocast(device_type='cuda', enabled=use_amp):
                logits = model(xb_noisy, t)
                loss = criterion(logits, yb)
            val_loss += loss.item() * xb.size(0)
            val_hits += (logits.argmax(1) == yb).sum().item()
    val_loss /= len(val_ds)
    val_acc = val_hits / len(val_ds)

    # Step scheduler after each batch in train loop
    scheduler.step(val_loss)
    curr_lr = optimizer.param_groups[0]['lr']
    print(f"[epoch:02d] train loss {tr_loss:.4f} acc {tr_acc:.3f} | "
          f"val loss {val_loss:.4f} acc {val_acc:.3f} | lr {curr_lr:.2e}")
    loss_history.append((tr_loss, val_loss))

    # --- early stop on val loss ---
    if val_loss < best_loss - 1e-6:
        best_loss = val_loss
        best_state = {k: v.detach().cpu().clone() for k, v in model.
↪state_dict().items()}
        print(f"\t*** New best! Val Loss: {val_loss:.4f} ***")
        epochs_no_improve = 0

```

```

else:
    epochs_no_improve += 1
    if epochs_no_improve >= PATIENCE:
        print(f"\nEarly stopping after {epochs_no_improve} epochs.")
        break

# load best
if best_state is not None:
    model.load_state_dict({k: v.to(device) for k, v in best_state.items()})

```

```

Epoch 1/1000: 100%|          | 9044/9044 [08:38<00:00, 17.45it/s]
[01] train loss 2.7791 acc 0.063 | val loss 2.7788 acc 0.070 | lr 3.00e-03
    *** New best! Val Loss: 2.7788 ***

Epoch 2/1000: 100%|          | 9044/9044 [07:53<00:00, 19.12it/s]
[02] train loss 2.6866 acc 0.100 | val loss 2.6023 acc 0.101 | lr 3.00e-03
    *** New best! Val Loss: 2.6023 ***

Epoch 3/1000: 100%|          | 9044/9044 [08:05<00:00, 18.62it/s]
[03] train loss 2.4708 acc 0.175 | val loss 2.4127 acc 0.191 | lr 3.00e-03
    *** New best! Val Loss: 2.4127 ***

Epoch 4/1000: 100%|          | 9044/9044 [08:24<00:00, 17.91it/s]
[04] train loss 2.3888 acc 0.207 | val loss 2.3984 acc 0.193 | lr 3.00e-03
    *** New best! Val Loss: 2.3984 ***

Epoch 5/1000: 100%|          | 9044/9044 [08:17<00:00, 18.17it/s]
[05] train loss 2.3389 acc 0.228 | val loss 2.3585 acc 0.217 | lr 3.00e-03
    *** New best! Val Loss: 2.3585 ***

Epoch 6/1000: 100%|          | 9044/9044 [08:03<00:00, 18.71it/s]
[06] train loss 2.3006 acc 0.242 | val loss 2.3120 acc 0.238 | lr 3.00e-03
    *** New best! Val Loss: 2.3120 ***

Epoch 7/1000: 100%|          | 9044/9044 [08:15<00:00, 18.24it/s]
[07] train loss 2.2704 acc 0.255 | val loss 2.3589 acc 0.205 | lr 3.00e-03

Epoch 8/1000: 100%|          | 9044/9044 [08:05<00:00, 18.65it/s]
[08] train loss 2.2496 acc 0.263 | val loss 2.2898 acc 0.248 | lr 3.00e-03
    *** New best! Val Loss: 2.2898 ***

Epoch 9/1000: 100%|          | 9044/9044 [08:09<00:00, 18.49it/s]
[09] train loss 2.2350 acc 0.269 | val loss 2.2623 acc 0.252 | lr 3.00e-03
    *** New best! Val Loss: 2.2623 ***

Epoch 10/1000: 100%|         | 9044/9044 [08:13<00:00, 18.32it/s]

```

```

[10] train loss 2.2176 acc 0.278 | val loss 2.2591 acc 0.247 | lr 3.00e-03
    *** New best! Val Loss: 2.2591 ***

Epoch 11/1000: 100%|      | 9044/9044 [08:22<00:00, 18.02it/s]

[11] train loss 2.2043 acc 0.282 | val loss 2.2519 acc 0.271 | lr 3.00e-03
    *** New best! Val Loss: 2.2519 ***

Epoch 12/1000: 100%|      | 9044/9044 [08:13<00:00, 18.34it/s]

[12] train loss 2.1940 acc 0.289 | val loss 2.2199 acc 0.269 | lr 3.00e-03
    *** New best! Val Loss: 2.2199 ***

Epoch 13/1000: 100%|      | 9044/9044 [08:17<00:00, 18.18it/s]

[13] train loss 2.1868 acc 0.291 | val loss 2.2016 acc 0.291 | lr 3.00e-03
    *** New best! Val Loss: 2.2016 ***

Epoch 14/1000: 100%|      | 9044/9044 [08:04<00:00, 18.68it/s]

[14] train loss 2.1758 acc 0.296 | val loss 2.2295 acc 0.282 | lr 3.00e-03

Epoch 15/1000: 100%|      | 9044/9044 [08:02<00:00, 18.73it/s]

[15] train loss 2.1698 acc 0.299 | val loss 2.1943 acc 0.285 | lr 3.00e-03
    *** New best! Val Loss: 2.1943 ***

Epoch 16/1000: 100%|      | 9044/9044 [07:51<00:00, 19.19it/s]

[16] train loss 2.1642 acc 0.301 | val loss 2.2051 acc 0.279 | lr 3.00e-03

Epoch 17/1000: 100%|      | 9044/9044 [07:58<00:00, 18.91it/s]

[17] train loss 2.1570 acc 0.304 | val loss 2.1834 acc 0.298 | lr 3.00e-03
    *** New best! Val Loss: 2.1834 ***

Epoch 18/1000: 100%|      | 9044/9044 [08:10<00:00, 18.44it/s]

[18] train loss 2.1477 acc 0.308 | val loss 2.1922 acc 0.286 | lr 3.00e-03

Epoch 19/1000: 100%|      | 9044/9044 [07:58<00:00, 18.88it/s]

[19] train loss 2.1463 acc 0.309 | val loss 2.1970 acc 0.286 | lr 3.00e-03

Epoch 20/1000: 100%|      | 9044/9044 [08:29<00:00, 17.73it/s]

[20] train loss 2.1440 acc 0.309 | val loss 2.1742 acc 0.293 | lr 3.00e-03
    *** New best! Val Loss: 2.1742 ***

Epoch 21/1000: 100%|      | 9044/9044 [08:07<00:00, 18.56it/s]

[21] train loss 2.1378 acc 0.314 | val loss 2.1616 acc 0.298 | lr 3.00e-03
    *** New best! Val Loss: 2.1616 ***

Epoch 22/1000: 100%|      | 9044/9044 [07:48<00:00, 19.29it/s]

[22] train loss 2.1353 acc 0.314 | val loss 2.1847 acc 0.298 | lr 3.00e-03

Epoch 23/1000: 100%|      | 9044/9044 [07:48<00:00, 19.29it/s]

[23] train loss 2.1300 acc 0.317 | val loss 2.1905 acc 0.293 | lr 3.00e-03

```


Epoch 24/1000: 100%| | 9044/9044 [07:46<00:00, 19.38it/s]
 [24] train loss 2.1274 acc 0.318 | val loss 2.1677 acc 0.301 | lr 2.40e-03
 Epoch 25/1000: 100%| | 9044/9044 [07:49<00:00, 19.28it/s]
 [25] train loss 2.1129 acc 0.324 | val loss 2.1637 acc 0.305 | lr 2.40e-03
 Epoch 26/1000: 100%| | 9044/9044 [07:54<00:00, 19.08it/s]
 [26] train loss 2.1074 acc 0.326 | val loss 2.1642 acc 0.297 | lr 2.40e-03
 Epoch 27/1000: 100%| | 9044/9044 [07:55<00:00, 19.00it/s]
 [27] train loss 2.1045 acc 0.327 | val loss 2.1475 acc 0.314 | lr 2.40e-03
 *** New best! Val Loss: 2.1475 ***
 Epoch 28/1000: 100%| | 9044/9044 [07:49<00:00, 19.26it/s]
 [28] train loss 2.1022 acc 0.328 | val loss 2.1551 acc 0.306 | lr 2.40e-03
 Epoch 29/1000: 100%| | 9044/9044 [07:52<00:00, 19.15it/s]
 [29] train loss 2.0993 acc 0.329 | val loss 2.1654 acc 0.298 | lr 2.40e-03
 Epoch 30/1000: 100%| | 9044/9044 [08:00<00:00, 18.81it/s]
 [30] train loss 2.0982 acc 0.330 | val loss 2.1686 acc 0.297 | lr 1.92e-03
 Epoch 31/1000: 100%| | 9044/9044 [07:49<00:00, 19.27it/s]
 [31] train loss 2.0862 acc 0.335 | val loss 2.1502 acc 0.307 | lr 1.92e-03
 Epoch 32/1000: 100%| | 9044/9044 [07:48<00:00, 19.31it/s]
 [32] train loss 2.0846 acc 0.335 | val loss 2.1377 acc 0.316 | lr 1.92e-03
 *** New best! Val Loss: 2.1377 ***
 Epoch 33/1000: 100%| | 9044/9044 [07:46<00:00, 19.38it/s]
 [33] train loss 2.0822 acc 0.337 | val loss 2.1510 acc 0.308 | lr 1.92e-03
 Epoch 34/1000: 100%| | 9044/9044 [07:46<00:00, 19.40it/s]
 [34] train loss 2.0795 acc 0.338 | val loss 2.1544 acc 0.302 | lr 1.92e-03
 Epoch 35/1000: 100%| | 9044/9044 [07:45<00:00, 19.44it/s]
 [35] train loss 2.0770 acc 0.340 | val loss 2.1604 acc 0.306 | lr 1.54e-03
 Epoch 36/1000: 100%| | 9044/9044 [07:45<00:00, 19.42it/s]
 [36] train loss 2.0705 acc 0.341 | val loss 2.1430 acc 0.308 | lr 1.54e-03
 Epoch 37/1000: 100%| | 9044/9044 [07:44<00:00, 19.45it/s]
 [37] train loss 2.0677 acc 0.343 | val loss 2.1618 acc 0.303 | lr 1.54e-03
 Epoch 38/1000: 100%| | 9044/9044 [07:48<00:00, 19.31it/s]
 [38] train loss 2.0619 acc 0.345 | val loss 2.1427 acc 0.311 | lr 1.23e-03
 Epoch 39/1000: 100%| | 9044/9044 [07:48<00:00, 19.30it/s]

[39] train loss 2.0605 acc 0.347 | val loss 2.1437 acc 0.310 | lr 1.23e-03
Epoch 40/1000: 100%| | 9044/9044 [07:51<00:00, 19.20it/s]

[40] train loss 2.0542 acc 0.349 | val loss 2.1353 acc 0.313 | lr 1.23e-03
*** New best! Val Loss: 2.1353 ***

Epoch 41/1000: 100%| | 9044/9044 [07:42<00:00, 19.54it/s]

[41] train loss 2.0563 acc 0.347 | val loss 2.1327 acc 0.316 | lr 1.23e-03
*** New best! Val Loss: 2.1327 ***

Epoch 42/1000: 100%| | 9044/9044 [07:43<00:00, 19.53it/s]

[42] train loss 2.0519 acc 0.349 | val loss 2.1320 acc 0.317 | lr 1.23e-03
*** New best! Val Loss: 2.1320 ***

Epoch 43/1000: 100%| | 9044/9044 [07:48<00:00, 19.30it/s]

[43] train loss 2.0511 acc 0.350 | val loss 2.1427 acc 0.313 | lr 1.23e-03

Epoch 44/1000: 100%| | 9044/9044 [07:50<00:00, 19.20it/s]

[44] train loss 2.0470 acc 0.352 | val loss 2.1350 acc 0.310 | lr 1.23e-03

Epoch 45/1000: 100%| | 9044/9044 [07:49<00:00, 19.28it/s]

[45] train loss 2.0463 acc 0.353 | val loss 2.1378 acc 0.310 | lr 9.83e-04

Epoch 46/1000: 100%| | 9044/9044 [07:47<00:00, 19.34it/s]

[46] train loss 2.0393 acc 0.356 | val loss 2.1269 acc 0.322 | lr 9.83e-04
*** New best! Val Loss: 2.1269 ***

Epoch 47/1000: 100%| | 9044/9044 [07:48<00:00, 19.32it/s]

[47] train loss 2.0385 acc 0.357 | val loss 2.1201 acc 0.322 | lr 9.83e-04
*** New best! Val Loss: 2.1201 ***

Epoch 48/1000: 100%| | 9044/9044 [07:47<00:00, 19.35it/s]

[48] train loss 2.0363 acc 0.357 | val loss 2.1335 acc 0.319 | lr 9.83e-04

Epoch 49/1000: 100%| | 9044/9044 [07:48<00:00, 19.32it/s]

[49] train loss 2.0351 acc 0.358 | val loss 2.1390 acc 0.310 | lr 9.83e-04

Epoch 50/1000: 100%| | 9044/9044 [07:48<00:00, 19.32it/s]

[50] train loss 2.0348 acc 0.358 | val loss 2.1239 acc 0.316 | lr 7.86e-04

Epoch 51/1000: 100%| | 9044/9044 [07:48<00:00, 19.31it/s]

[51] train loss 2.0280 acc 0.360 | val loss 2.1272 acc 0.317 | lr 7.86e-04

Epoch 52/1000: 100%| | 9044/9044 [07:49<00:00, 19.27it/s]

[52] train loss 2.0282 acc 0.362 | val loss 2.1304 acc 0.314 | lr 7.86e-04

Epoch 53/1000: 100%| | 9044/9044 [07:49<00:00, 19.28it/s]

[53] train loss 2.0260 acc 0.363 | val loss 2.1274 acc 0.314 | lr 6.29e-04

```

Epoch 54/1000: 100%|          | 9044/9044 [07:47<00:00, 19.35it/s]
[54] train loss 2.0217 acc 0.364 | val loss 2.1219 acc 0.319 | lr 6.29e-04
Epoch 55/1000: 100%|          | 9044/9044 [07:48<00:00, 19.31it/s]
[55] train loss 2.0240 acc 0.361 | val loss 2.1188 acc 0.322 | lr 6.29e-04
      *** New best! Val Loss: 2.1188 ***
Epoch 56/1000: 100%|          | 9044/9044 [07:50<00:00, 19.24it/s]
[56] train loss 2.0187 acc 0.365 | val loss 2.1275 acc 0.316 | lr 6.29e-04
Epoch 57/1000: 100%|          | 9044/9044 [07:47<00:00, 19.34it/s]
[57] train loss 2.0165 acc 0.365 | val loss 2.1237 acc 0.320 | lr 6.29e-04
Epoch 58/1000: 100%|          | 9044/9044 [07:49<00:00, 19.26it/s]
[58] train loss 2.0180 acc 0.365 | val loss 2.1323 acc 0.314 | lr 5.03e-04
Epoch 59/1000: 100%|          | 9044/9044 [07:53<00:00, 19.11it/s]
[59] train loss 2.0144 acc 0.366 | val loss 2.1253 acc 0.322 | lr 5.03e-04
Epoch 60/1000: 100%|          | 9044/9044 [08:06<00:00, 18.58it/s]
[60] train loss 2.0117 acc 0.369 | val loss 2.1221 acc 0.317 | lr 5.03e-04
Epoch 61/1000: 100%|          | 9044/9044 [08:02<00:00, 18.75it/s]
[61] train loss 2.0125 acc 0.367 | val loss 2.1268 acc 0.316 | lr 4.03e-04
Epoch 62/1000: 100%|          | 9044/9044 [08:16<00:00, 18.21it/s]
[62] train loss 2.0102 acc 0.368 | val loss 2.1287 acc 0.318 | lr 4.03e-04
Epoch 63/1000: 100%|          | 9044/9044 [07:58<00:00, 18.90it/s]
[63] train loss 2.0089 acc 0.369 | val loss 2.1219 acc 0.323 | lr 4.03e-04
Epoch 64/1000: 100%|          | 9044/9044 [08:21<00:00, 18.05it/s]
[64] train loss 2.0084 acc 0.370 | val loss 2.1258 acc 0.320 | lr 3.22e-04
Epoch 65/1000: 100%|          | 9044/9044 [08:02<00:00, 18.75it/s]
[65] train loss 2.0051 acc 0.370 | val loss 2.1332 acc 0.316 | lr 3.22e-04

```

Early stopping after 10 epochs.

```

[16]: # @title Test-Set Report
model.eval()
preds, trues = [], []
with torch.no_grad():
    for x, y in tqdm(test_dl, desc="Test inference"):
        x, y = x.to(device), y.to(device)
        t0 = torch.zeros(x.size(0), device=device, dtype=torch.long)
        logits = model(x, t0)

```

```

        preds.extend(logits.argmax(1).cpu().numpy())
        trues.extend(y.cpu().numpy())

# Get unique labels present in the test set
unique_labels = sorted(list(set(trues)))

# Filter and order target_names based on unique_labels
target_names = [IDX2PHASE[i] for i in sorted(set(trues))]

report = classification_report(trues, preds, labels=unique_labels,
                               target_names=target_names, digits=3)
print(report)
cm = confusion_matrix(trues, preds, labels=unique_labels)
print("Confusion matrix\n", cm)

# loss history plot
train_losses, val_losses = zip(*loss_history)
plt.figure(figsize=(8,5))
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Epochs')
plt.legend()
plt.show()

```

```

Test inference: 100%|          | 1145/1145 [00:23<00:00, 48.40it/s]
/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

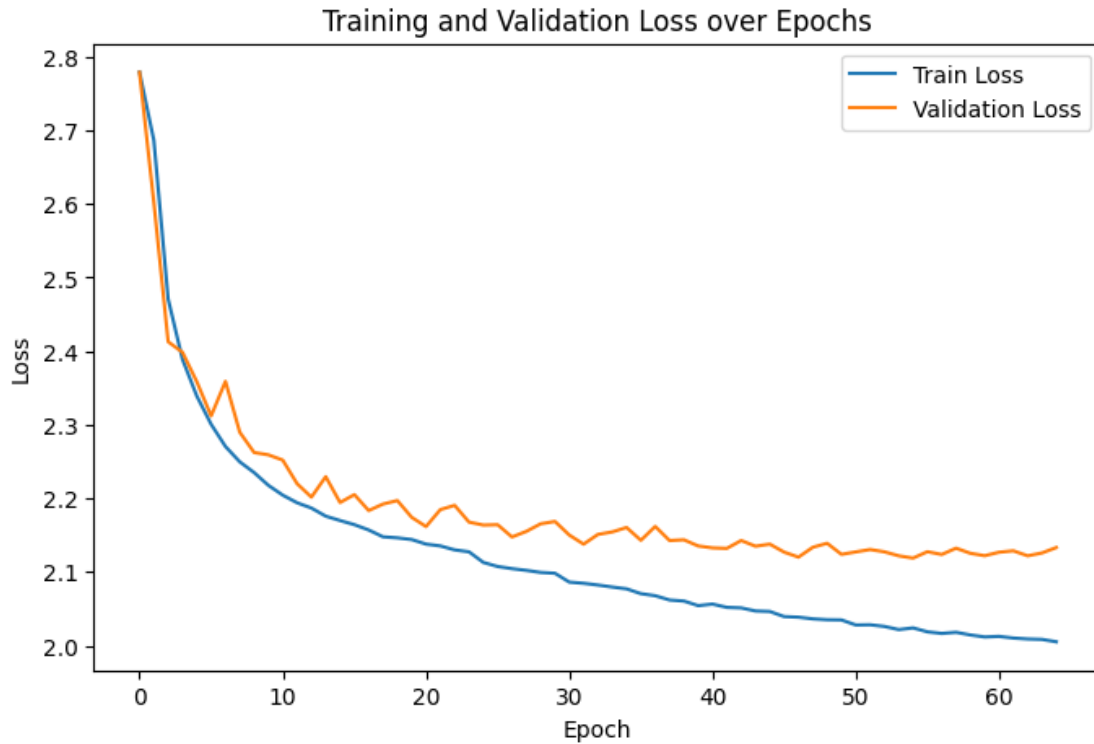
	precision	recall	f1-score	support
tPB2	0.613	0.803	0.695	2940
tPNa	0.679	0.776	0.724	3283

tPNf	0.653	0.551	0.598	3261
t2	0.804	0.810	0.807	3395
t3	0.391	0.557	0.459	1078
t4	0.747	0.543	0.629	3282
t5	0.383	0.403	0.393	1839
t6	0.382	0.329	0.354	2030
t7	0.342	0.379	0.360	1932
t8	0.635	0.486	0.551	2919
t9+	0.619	0.592	0.605	2870
tM	0.587	0.760	0.662	2562
tSB	0.654	0.497	0.565	2324
tB	0.527	0.659	0.586	1561
tEB	0.677	0.588	0.630	1260
tHB	0.000	0.000	0.000	94
accuracy			0.600	36630
macro avg	0.543	0.546	0.539	36630
weighted avg	0.607	0.600	0.597	36630

Confusion matrix

[[2361	364	207	2	4	0	1	0	0	0	0	1	0	0
0	0]												
[502	2549	199	10	0	0	0	0	0	0	0	22	1	0
0	0]												
[684	581	1797	184	1	1	0	0	0	0	0	13	0	0
0	0]												
[77	55	147	2751	232	42	30	7	0	1	2	46	4	1
0	0]												
[14	11	7	293	600	43	47	16	8	1	20	18	0	0
0	0]												
[20	6	53	120	519	1782	526	125	42	8	32	47	2	0
0	0]												
[15	0	34	11	86	293	742	375	157	18	59	49	0	0
0	0]												
[32	19	40	38	73	92	366	668	391	118	109	76	6	2
0	0]												
[28	20	50	2	8	74	137	345	732	329	137	63	6	1
0	0]												
[22	23	4	3	6	50	46	184	681	1419	394	72	15	0
0	0]												
[27	31	41	2	1	8	14	21	116	335	1698	509	65	2
0	0]												
[25	30	41	2	3	1	25	7	12	5	160	1947	280	24
0	0]												
[24	32	42	1	0	0	1	0	1	0	118	439	1156	490
20	0]												
[21	33	43	0	0	0	0	0	0	0	2	7	175	1029
251	0]												

```
[ 0  2 45  1  0  0  0  1  0  0  2 10 56 402
741 0]
[ 0  0  2  0  1  0  0  0  0  0  8  0  1  0
82  0]]
```



```
[17]: # @title Save Model
import datetime, torch
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = f"embryo_phase_models/phase_classifier_{timestamp}.pth"
torch.save(model, out_path)
print("Model saved to", out_path)
```

Model saved to embryo_phase_models/phase_classifier_20260118_010706.pth